

SDE Portfolio

Complete Build Reference

[API Specs](#) · [DB Schemas](#) · [Packages](#) · [Env Vars](#) · [Folder Structure](#) · [Build Checklist](#)

Karanam Sai Manikanta

• PROJECT 1

Realtime Collaborative Code Judge

[React](#) · [Node.js](#) · [MongoDB](#) · [Redis](#) · [WebSockets](#) · [Docker](#)

• PROJECT 2

Distributed Job Scheduler Engine

[Spring Boot](#) · [PostgreSQL](#) · [Redis](#) · [React](#)

• PROJECT 3

API Gateway with Rate Limiting & Analytics

[ASP.NET Core](#) · [Angular](#) · [SQL Server](#) · [Redis](#) · [SignalR](#)

PROJECT 1 — Realtime Collaborative Code Judge

Two users solve the same problem simultaneously with live cursor sync, real-time output, and Docker-sandboxed code execution.

1.1 Packages to Install

Backend — Node.js (npm)

Package	Version	Purpose
<code>express</code>	<code>^4.18.2</code>	HTTP server and routing
<code>mongoose</code>	<code>^8.0.0</code>	MongoDB ODM for schemas and queries
<code>ws</code>	<code>^8.16.0</code>	WebSocket server (lighter than Socket.IO)
<code>redis</code>	<code>^4.6.10</code>	Redis client — pub/sub + session state
<code>ioredis</code>	<code>^5.3.2</code>	Alternative Redis client with better pub/sub API
<code>jsonwebtoken</code>	<code>^9.0.2</code>	JWT sign and verify
<code>bcryptjs</code>	<code>^2.4.3</code>	Password hashing
<code>dockerode</code>	<code>^4.0.0</code>	Control Docker containers from Node.js
<code>uuid</code>	<code>^9.0.0</code>	Generate unique session/submission IDs
<code>dotenv</code>	<code>^16.3.1</code>	Load .env variables
<code>cors</code>	<code>^2.8.5</code>	Cross-origin headers
<code>helmet</code>	<code>^7.1.0</code>	Security headers middleware
<code>express-rate-limit</code>	<code>^7.1.5</code>	Rate limit execution endpoint
<code>joi</code>	<code>^17.11.0</code>	Request body validation
<code>morgan</code>	<code>^1.10.0</code>	HTTP request logging

```
# Install all backend packages
npm install express mongoose ws redis ioredis jsonwebtoken bcryptjs \
  dockerode uuid dotenv cors helmet express-rate-limit joi morgan

# Dev dependencies
npm install -D nodemon jest supertest @types/node
```

Frontend — React (npm)

Package	Version	Purpose
<code>@monaco-editor/react</code>	<code>^4.6.0</code>	VS Code-grade code editor in browser
<code>axios</code>	<code>^1.6.0</code>	HTTP client for API calls
<code>react-router-dom</code>	<code>^6.20.0</code>	Client-side routing
<code>zustand</code>	<code>^4.4.7</code>	Lightweight state management

recharts	^2.10.0	Verdict timeline charts
react-hot-toast	^2.4.1	Toast notifications for verdicts
tailwindcss	^3.3.6	Utility-first CSS
lucide-react	^0.303.0	Icon library
date-fns	^3.0.0	Format submission timestamps

```
# Create React app
npx create-react-app code-judge-client --template typescript
cd code-judge-client

npm install @monaco-editor/react axios react-router-dom zustand \
    recharts react-hot-toast lucide-react date-fns
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

Docker requirement: Install Docker Desktop locally. The backend uses dockerode to spin up isolated containers per code execution. Test with: `docker pull node:18-alpine` && `docker pull python:3.11-alpine`

1.2 Environment Variables (.env)

Variable	Example Value	Description
PORT	5000	Express server port
MONGODB_URI	mongodb:// localhost:27017/ codejudge	MongoDB connection string
REDIS_URL	redis:// localhost:6379	Redis connection URL
JWT_SECRET	your_super_secret_key _here	JWT signing secret — min 32 chars
JWT_EXPIRES_IN	7d	JWT expiry duration
DOCKER_SOCKET	/var/run/docker.sock	Docker socket path for dockerode
CODE_EXEC_TIMEOUT	5000	Max execution time in ms per submission
CODE_EXEC_MEMORY	128m	Max memory per Docker container
CLIENT_URL	http://localhost:3000	Frontend URL for CORS whitelist

1.3 Folder Structure

```
code-judge/
├── server/
│   ├── src/
│   │   ├── config/
│   │   │   ├── db.js          ← MongoDB connection
│   │   │   └── redis.js       ← Redis client setup
│   │   ├── middleware/
│   │   │   ├── auth.js       ← JWT verify middleware
│   │   │   ├── ratelimiter.js ← Execution rate limiter
│   │   │   └── validate.js    ← Joi schema validator
│   │   └── models/
```



1.4 Endpoint Overview

Method	Endpoint	Auth	Description
POST	/api/auth/register	Public	Register new user
POST	/api/auth/login	Public	Login and receive JWT
GET	/api/auth/me	JWT	Get current user profile
GET	/api/problems	Public	List problems (paginated, filterable)
GET	/api/problems/:id	Public	Get full problem with test cases
POST	/api/problems	JWT	Create problem (admin)
PUT	/api/problems/:id	JWT	Update problem (admin)
DELETE	/api/problems/:id	JWT	Delete problem (admin)
POST	/api/sessions	JWT	Create collaborative session
GET	/api/sessions/:sessionId	JWT	Get session details + current code
POST	/api/sessions/:sessionId/join	JWT	Join session by code

DELETE	/api/sessions/:sessionId	JWT	End session (host only)
POST	/api/execute	JWT	Execute code in Docker sandbox
GET	/api/submissions	JWT	My submission history
GET	/api/submissions/:id	JWT	Single submission result + test details
GET	/api/leaderboard/:problemId	Public	Top submissions for a problem
GET	/api/health	Public	Server + Redis + DB health

WebSocket events handled on same server via ws://. Connect with ?token=JWT in query param. See Section 1.7

1.5 Auth Endpoints

[POST] /api/auth/register *Create new user account*

Auth: None

Request Body:

```
{
  "username": "sai_dev",
  "email": "sai@example.com",
  "password": "StrongPass123!"
}
```

Response 201:

```
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIs... ",
  "user": { "_id": "64f3a...", "username": "sai_dev", "email": "sai@example.com", "role": "user" }
}
```

Errors: 400 Email already registered | 422 Validation failed

[POST] /api/auth/login *Authenticate and receive JWT*

Auth: None

Request Body: { "email": "sai@example.com", "password": "StrongPass123!" }

Response 200: { "success": true, "token": "eyJ...", "expiresIn": "7d" }

Errors: 401 Invalid credentials | 404 User not found

[GET] /api/auth/me *Get authenticated user profile*

Auth: JWT Bearer Token required

Response 200: { "_id": "...", "username": "sai_dev", "stats": { "totalSubmissions": 45, "solved": 18, "streak": 7 } }

1.6 Problem Endpoints

[GET] /api/problems *List problems with filters*

Query Params: ?page=1&limit=20&difficulty=easy&tag=arrays&search=two+sum

Response 200:

```
{
```

```

"problems": [
  { "_id": "64f3b...", "title": "Two Sum", "slug": "two-sum", "difficulty": "easy",
    "tags": ["arrays", "hash-map"], "totalSubmissions": 1240, "acceptanceRate": 72.4 }
],
"pagination": { "page": 1, "limit": 20, "total": 150, "pages": 8 }
}

```

| **[GET] /api/problems/:id** *Full problem with test cases and starter code*

Response 200:

```

{
  "_id": "64f3b...", "title": "Two Sum", "difficulty": "easy",
  "description": "Given an array of integers nums...",
  "examples": [{ "input": "nums=[2,7,11,15], target=9", "output": "[0,1]",
    "explanation": "..."} ],
  "starterCode": {
    "javascript": "function twoSum(nums, target) {\n // your code\n}",
    "python":      "def two_sum(nums, target):\n    pass",
    "java":        "public int[] twoSum(int[] nums, int target) {\n}"
  },
  "testCases": [{ "input": "[2,7,11,15]\n9", "expectedOutput": "[0,1]", "isHidden": false }],
  "timeLimit": 2000, "memoryLimit": 256
}

```

| **[POST] /api/problems** *Create problem (admin only)*

Auth: JWT + role: admin

Request Body: Full problem object: title, description, difficulty, tags, examples, starterCode, testCases, timeLimit, memoryLimit

Response 201: { "success": true, "problem": { "_id": "...", "slug": "two-sum" } }

1.7 Collaborative Session Endpoints

Core feature: Sessions are created by a host, joined by a guest. Both users share a live editor via WebSocket. Session state (current code, cursors, chat) lives in Redis.

| **[POST] /api/sessions** *Create collaborative session*

Request Body: { "problemId": "64f3b...", "language": "javascript", "isPublic": false }

Response 201:

```

{
  "sessionId": "sess_xK9mP2qR",
  "joinCode": "XK9M",
  "problem": { "_id": "...", "title": "Two Sum" },
  "host": { "_id": "...", "username": "sai_dev" },
  "wsUrl": "ws://localhost:5000/session/sess_xK9mP2qR",
  "expiresAt": "2025-01-01T11:00:00Z"
}

```

| **[POST] /api/sessions/:sessionId/join** *Join existing session*

Request Body: { "joinCode": "XK9M" }

Response 200: Session object + currentCode + all participants + wsUrl

Errors: 404 Session not found | 403 Session full | 400 Wrong join code

| **[GET] /api/sessions/:sessionId** *Get live session state*

Response 200: `currentCode`, `participants[]`, `verdictHistory[]`, `chatMessages[]`, `status`

1.8 Code Execution Endpoint

Security note: Always run in Docker with `--memory=128m --cpus=0.5 --network=none` flags. Kill container after `timeLimit` ms using `dockerode.container.stop()`

| **[POST] /api/execute** *Execute code in Docker sandbox*

Auth: JWT required | Rate limited: 10 executions/minute per user

Request Body:

```
{
  "sessionId": "sess_xK9mP2qR",
  "problemId": "64f3b...",
  "code": "function twoSum(nums, target) { ... }",
  "language": "javascript",
  "runMode": "run"
}
```

runMode: "run" = sample test cases only | "submit" = all cases including hidden

Response 200 (Accepted):

```
{
  "submissionId": "sub_7tRp...",
  "verdict": "Accepted",
  "runtime": "48ms", "memory": "42.1MB",
  "passedCases": 12, "totalCases": 12,
  "testResults": [
    { "caseIndex": 0, "status": "Passed", "input": "[2,7,11,15]\n9",
      "expected": "[0,1]", "actual": "[0,1]", "runtime": "2ms" }
  ]
}
```

Response 200 (TLE): `{ "verdict": "Time Limit Exceeded", "runtime": ">2000ms", "passedCases": 8, "totalCases": 12 }`

Possible verdicts: Accepted | Wrong Answer | Time Limit Exceeded | Memory Limit Exceeded | Runtime Error | Compilation Error

1.9 WebSocket Events

Client → Server

```
{ "event": "code:change", "payload": { "code": "...", "cursorPosition": { "line": 5, "col": 12 } } }
{ "event": "code:submit", "payload": { "code": "...", "language": "javascript" } }
{ "event": "chat:message", "payload": { "message": "check line 8" } }
{ "event": "session:leave" }
```

Server → Client

```
{ "event": "code:update", "payload": { "code": "...", "from": "ravi_123", "cursorPosition": { ... } } }
{ "event": "verdict:result", "payload": { "verdict": "Accepted", "runtime": "48ms", "passedCases": 12 } }
{ "event": "session:joined", "payload": { "user": { "username": "ravi_123" } } }
{ "event": "session:left", "payload": { "username": "ravi_123" } }
{ "event": "chat:message", "payload": { "from": "sai_dev", "message": "check line 8", "ts": "..." } }
```

1.10 MongoDB Schemas

User Schema

Field	Type	Required	Description
_id	ObjectId	Yes	Auto-generated MongoDB ID
username	String	Yes	Unique, lowercase, 3-20 chars, indexed
email	String	Yes	Unique, indexed
password	String	Yes	Bcrypt hashed, never returned in responses
role	String	Yes	Enum: user admin. Default: user
stats.totalSubmissions	Number	No	Cached count, incremented on submit
stats.solved	Number	No	Unique accepted problems count
stats.streak	Number	No	Current daily solve streak
createdAt	Date	Yes	Auto via timestamps: true

Problem Schema

Field	Type	Required	Description
_id	ObjectId	Yes	Auto-generated
title	String	Yes	Unique problem title
slug	String	Yes	URL-friendly unique identifier, indexed
description	String	Yes	Full markdown problem statement
difficulty	String	Yes	Enum: easy medium hard
tags	[String]	No	e.g. [arrays, hash-map, dp]
examples	[Object]	Yes	{ input, output, explanation }
starterCode	Object	Yes	{ javascript, python, java }
testCases	[Object]	Yes	{ input, expectedOutput, isHidden }
timeLimit	Number	No	ms. Default: 2000
memoryLimit	Number	No	MB. Default: 256
totalSubmissions	Number	No	Cached, incremented on submit
acceptanceRate	Number	No	Recomputed after each submission
createdBy	ObjectId	Yes	Ref: User (admin)

Session Schema

Field	Type	Required	Description
_id	ObjectId	Yes	Auto-generated
sessionId	String	Yes	Custom short ID e.g. sess_xK9mP2qR, indexed
joinCode	String	Yes	4-char code for joining
problemId	ObjectId	Yes	Ref: Problem
host	ObjectId	Yes	Ref: User
guest	ObjectId	No	Ref: User (null until joined)
language	String	Yes	javascript python java
currentCode	String	No	Snapshot saved from Redis periodically
status	String	Yes	waiting active ended
verdictHistory	[Object]	No	{ verdict, runtime, submittedAt, by }
expiresAt	Date	Yes	Auto-expire after 1hr inactivity

Submission Schema

Field	Type	Required	Description
_id	ObjectId	Yes	Auto-generated
userId	ObjectId	Yes	Ref: User, indexed
problemId	ObjectId	Yes	Ref: Problem, indexed
sessionId	String	No	Set if submitted in a session
code	String	Yes	Full submitted source code
language	String	Yes	javascript python java
verdict	String	Yes	Accepted Wrong Answer TLE MLE Runtime Error
runtime	String	No	e.g. 48ms
memory	String	No	e.g. 42.1MB
passedCases	Number	Yes	Test cases passed
totalCases	Number	Yes	Total test cases run
submittedAt	Date	Yes	Auto timestamp

Redis Key Map

session:{sessionId}:code	→ STRING	Live editor code (synced in real-time)
session:{sessionId}:participants	→ SET	Connected user IDs
session:{sessionId}:cursors	→ HASH	{ userId: { line, col } }
session:{sessionId}:chat	→ LIST	Last 50 messages (LPUSH + LTRIM 0 49)
exec:lock:{sessionId}	→ STRING	Prevents concurrent execution (TTL: 10s)
rate:exec:{userId}	→ STRING	Execution rate limit counter (TTL: 60s)

1.11 Build Checklist

Task	Priority	Status
☐ Setup MongoDB + Redis locally (Docker Compose recommended)	High	Not Started
☐ Scaffold Express server with auth routes + JWT middleware	High	Not Started
☐ Build Problem CRUD endpoints + MongoDB schema	High	Not Started
☐ Implement WebSocket server (ws library) + Redis pub/sub sync	High	Not Started
☐ Build Docker execution service (dockerode) for JS + Python	High	Not Started
☐ Implement Judge service — run code + compare expected output	High	Not Started
☐ Build Session create/join/end endpoints + Redis state	High	Not Started
☐ Build React Monaco editor with WebSocket live sync	High	Not Started
☐ Add cursor overlay (show partner's position in editor)	Medium	Not Started
☐ Build verdict timeline chart (Recharts)	Medium	Not Started
☐ Add execution rate limiter (express-rate-limit + Redis)	Medium	Not Started
☐ Add Java support to Docker judge	Medium	Not Started
☐ Leaderboard endpoint + UI	Low	Not Started
☐ Session chat panel	Low	Not Started
☐ Deploy: Render (backend) + Vercel (frontend) + Railway (Redis)	Low	Not Started

PROJECT 2 — Distributed Job Scheduler Engine

A mini AWS EventBridge — schedule cron jobs, async workers, retry logic, dead letter queues, and priority queues with a React dashboard.

2.1 Packages to Install

Backend — Spring Boot (Maven pom.xml)

Package	Version	Purpose
spring-boot-starter-web	3.2.x	REST API controllers and embedded Tomcat
spring-boot-starter-data-jpa	3.2.x	JPA/Hibernate ORM for PostgreSQL
spring-boot-starter-security	3.2.x	JWT-based auth security chain
spring-boot-starter-validation	3.2.x	Bean validation (@Valid, @NotBlank)
spring-boot-starter-actuator	3.2.x	Health check and metrics endpoints
postgresql	42.7.x	PostgreSQL JDBC driver
spring-boot-starter-data-redis	3.2.x	Redis template + pub/sub
jedis	5.1.x	Redis client used by Spring Data Redis
jjwt-api	0.12.x	JWT creation and parsing (JJWT library)
jjwt-impl	0.12.x	JJWT implementation (runtime scope)
jjwt-jackson	0.12.x	JJWT Jackson serializer (runtime scope)
quartz-scheduler	2.3.x	Cron expression parsing and job scheduling
okhttp	4.12.x	HTTP client for executing HTTP-type jobs
flyway-core	10.x	Database migration versioning
lombok	1.18.x	Reduce boilerplate (@Getter, @Builder, etc.)
mapstruct	1.5.x	DTO ↔ Entity mapping
springdoc-openapi-starter-webmvc-ui	2.3.x	Swagger UI auto-generated from annotations

```
<!-- Add to pom.xml <dependencies> -->
<dependency><groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId></dependency>
<dependency><groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId></dependency>
<dependency><groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId></dependency>
<dependency><groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-redis</artifactId></dependency>
<dependency><groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId><version>0.12.3</version></dependency>
```

```
<dependency><groupId>org.quartz-scheduler</groupId>
  <artifactId>quartz</artifactId><version>2.3.2</version></dependency>
<dependency><groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId><optional>true</optional></dependency>
```

Frontend — React (npm)

Package	Version	Purpose
axios	<code>^1.6.0</code>	API calls to Spring Boot backend
react-router-dom	<code>^6.20.0</code>	Page routing
zustand	<code>^4.4.7</code>	Global state (jobs, executions)
recharts	<code>^2.10.0</code>	Execution timeline + success rate charts
react-hot-toast	<code>^2.4.1</code>	Job trigger success/failure toasts
react-query	<code>^5.13.4</code>	Server state caching + auto-refetch
date-fns	<code>^3.0.0</code>	Format nextRunAt, lastRunAt timestamps
cronstrue	<code>^2.50.0</code>	Convert cron expr to human text e.g. 'Every day at 9AM'
tailwindcss	<code>^3.3.6</code>	Utility CSS
lucide-react	<code>^0.303.0</code>	Icons (play, pause, trash, clock)

```
npm install axios react-router-dom zustand recharts react-hot-toast \
  @tanstack/react-query date-fns cronstrue lucide-react
npm install -D tailwindcss postcss autoprefixer
```

2.2 Environment Variables (application.properties)

Variable	Example Value	Description
server.port	<code>8080</code>	Spring Boot server port
spring.datasource.url	<code>jdbc:postgresql://localhost:5432/scheduler</code>	PostgreSQL JDBC URL
spring.datasource.username	<code>postgres</code>	DB username
spring.datasource.password	<code>yourpassword</code>	DB password
spring.jpa.hibernate.ddl-auto	<code>validate</code>	Use Flyway for schema, not auto-create
spring.data.redis.host	<code>localhost</code>	Redis host
spring.data.redis.port	<code>6379</code>	Redis port
jwt.secret	<code>your_256bit_secret_key_here</code>	JWT HMAC-SHA256 secret
jwt.expiration	<code>604800000</code>	JWT expiry in ms (7 days)
job.executor.thread-pool-size	<code>10</code>	ExecutorService thread pool size

<code>job.http.default-timeout-ms</code>	<code>30000</code>	Default HTTP job timeout
<code>job.scheduler.poll-interval-ms</code>	<code>5000</code>	How often scheduler checks next_run_at
<code>cors.allowed-origins</code>	<code>http://localhost:3000</code>	Frontend URL

2.3 Folder Structure

Backend — Spring Boot

```

job-scheduler-server/
├── src/main/java/com/sai/scheduler/
│   ├── SchedulerApplication.java          ← @SpringBootApplication entry
│   └── config/
│       ├── SecurityConfig.java           ← JWT filter chain, CORS, permit rules
│       ├── RedisConfig.java              ← RedisTemplate bean setup
│       ├── ExecutorConfig.java           ← ThreadPoolTaskExecutor bean
│       └── OpenApiConfig.java             ← Swagger config + JWT bearer scheme
│   ├── security/
│       ├── JwtTokenProvider.java          ← generate, validate, parse JWT
│       ├── JwtAuthFilter.java             ← OncePerRequestFilter extracts JWT
│       └── UserDetailsServiceImpl.java    ← loadUserByUsername from DB
│   ├── entity/
│       ├── User.java                     ← @Entity users table
│       ├── Job.java                       ← @Entity jobs table
│       └── Execution.java                  ← @Entity executions table
│   ├── repository/
│       ├── UserRepository.java            ← JpaRepository<User, Long>
│       ├── JobRepository.java             ← Custom queries: findByNextRunAtBefore()
│       └── ExecutionRepository.java        ← findByJobIdOrderByCreatedAtDesc()
│   ├── dto/
│       ├── request/
│           ├── CreateJobRequest.java
│           ├── UpdateJobRequest.java
│           └── LoginRequest.java
│       └── response/
│           ├── JobResponse.java
│           ├── ExecutionResponse.java
│           └── DashboardStatsResponse.java
│   ├── controller/
│       ├── AuthController.java
│       ├── JobController.java
│       ├── ExecutionController.java
│       ├── QueueController.java
│       └── DashboardController.java
│   ├── service/
│       ├── AuthService.java
│       ├── JobService.java                ← CRUD + schedule computation
│       ├── ExecutionService.java          ← Create, update status, retry logic
│       ├── QueueService.java              ← Redis LPUSH/RPOP queue operations
│       ├── SchedulerService.java          ← Polls DB for due jobs, enqueues them
│       ├── WorkerService.java             ← @Async picks jobs from queue, executes
│       └── HttpJobExecutor.java           ← OkHttp calls for HTTP-type jobs
│   └── scheduler/

```



Frontend — React



2.4 Endpoint Overview

Method	Endpoint	Auth	Description
POST	/api/auth/register	Public	Register user
POST	/api/auth/login	Public	Login and receive JWT
POST	/api/jobs	JWT	Create a new scheduled job
GET	/api/jobs	JWT	List all jobs (paginated)

GET	/api/jobs/:id	JWT	Get single job with full config
PUT	/api/jobs/:id	JWT	Update job config or schedule
DELETE	/api/jobs/:id	JWT	Delete job permanently
POST	/api/jobs/:id/trigger	JWT	Manually trigger job now
POST	/api/jobs/:id/pause	JWT	Pause scheduled job
POST	/api/jobs/:id/resume	JWT	Resume paused job
GET	/api/jobs/:id/executions	JWT	Execution history for job
GET	/api/executions/:execId	JWT	Single execution with logs
POST	/api/executions/:execId/retry	JWT	Retry a failed execution
GET	/api/queues	JWT	All queues with depth + stats
GET	/api/queues/:name/jobs	JWT	Pending jobs in a queue
DELETE	/api/queues/:name/flush	JWT	Flush all pending from queue
GET	/api/dashboard/stats	JWT	System-wide stats for dashboard
GET	/api/health	Public	Health check: DB + Redis status

2.5 Job Endpoints

| [POST] /api/jobs *Create a new scheduled job*

Auth: JWT required

Request Body:

```
{
  "name": "Daily Report Email",
  "type": "HTTP",
  "queue": "emails",
  "schedule": { "type": "cron", "expression": "0 9 * * *" },
  "payload": {
    "url": "https://myapp.com/api/send-report",
    "method": "POST",
    "headers": { "Authorization": "Bearer token" },
    "body": { "reportType": "daily" }
  },
  "retryPolicy": { "maxRetries": 3, "backoffType": "exponential", "initialDelayMs": 1000 },
  "timeout": 30000,
  "priority": "normal"
}
```

schedule.type: "cron" | "interval" (ms) | "once" (ISO timestamp)

priority: "low" | "normal" | "high" | "critical"

Response 201: { "id": 42, "status": "active", "nextRunAt": "2025-01-02T09:00:00Z" }

| [POST] /api/jobs/:id/trigger *Manually trigger job immediately*

Request Body: { "overridePayload": {} } — optional

Response 200: { "executionId": "exec_8pQr...", "status": "queued", "queuedAt": "..." }

| [POST] /api/jobs/:id/pause *Pause job*

Response 200: { "id": 42, "status": "paused", "pausedAt": "..." }

| **[POST] /api/jobs/:id/resume** *Resume paused job*

Response 200: { "id": 42, "status": "active", "nextRunAt": "..." }

2.6 Execution Endpoints

| **[GET] /api/executions/:execId** *Single execution with full logs*

Response 200:

```
{
  "id": "exec_8pQr...", "jobId": 42, "status": "failed", "attempt": 2,
  "startedAt": "2025-01-01T09:00:00Z", "completedAt": "2025-01-01T09:00:31Z",
  "durationMs": 30502,
  "error": { "message": "Connection timeout", "code": "ETIMEDOUT" },
  "logs": [
    { "ts": "...", "level": "info", "msg": "Execution started" },
    { "ts": "...", "level": "error", "msg": "Request timed out after 30000ms" }
  ],
  "nextRetryAt": "2025-01-01T09:00:33Z", "retriesLeft": 1
}
```

| **[POST] /api/executions/:execId/retry** *Manually retry failed execution*

Response 200: { "newExecutionId": "exec_9sRt...", "status": "queued" }

| **[GET] /api/dashboard/stats** *System stats for React dashboard*

Response 200:

```
{
  "totalJobs": 15, "activeJobs": 12, "pausedJobs": 2,
  "executionsToday": 148, "successRate": 97.3, "avgDurationMs": 1240,
  "upcomingJobs": [{ "id": 42, "name": "Daily Report", "nextRunAt": "..." }],
  "recentFailures": [{ "execId": "...", "jobName": "...", "error": "..." }],
  "executionTimeline": [
    { "hour": "09:00", "success": 12, "failed": 0 },
    { "hour": "10:00", "success": 18, "failed": 1 }
  ]
}
```

2.7 PostgreSQL Schemas

users

Field	Type	Required	Description
id	BIGSERIAL	Yes	Primary key, auto-increment
name	VARCHAR(100)	Yes	Full name
email	VARCHAR(255)	Yes	Unique, indexed
password	VARCHAR(255)	Yes	BCrypt hashed

role	VARCHAR(20)	Yes	user admin. Default: user
created_at	TIMESTAMP	Yes	DEFAULT NOW()
updated_at	TIMESTAMP	Yes	Updated on save

jobs

Field	Type	Required	Description
id	BIGSERIAL	Yes	Primary key
user_id	BIGINT	Yes	FK → users.id, indexed
name	VARCHAR(200)	Yes	Job display name
type	VARCHAR(20)	Yes	HTTP FUNCTION
queue	VARCHAR(100)	Yes	Target queue name. Default: default
status	VARCHAR(20)	Yes	active paused deleted
schedule_type	VARCHAR(20)	Yes	cron interval once
schedule_expr	VARCHAR(100)	Yes	Cron expression or interval ms
payload	JSONB	Yes	Job-specific config (url, method, body)
retry_policy	JSONB	No	{ maxRetries, backoffType, initialDelayMs }
timeout	INT	No	ms. Default: 30000
priority	VARCHAR(20)	No	low normal high critical
last_run_at	TIMESTAMP	No	Last execution start time
next_run_at	TIMESTAMP	No	Next scheduled run, indexed
total_runs	INT	No	Cached count. Default: 0
success_count	INT	No	Cached success count
failure_count	INT	No	Cached failure count
created_at	TIMESTAMP	Yes	DEFAULT NOW()
updated_at	TIMESTAMP	Yes	Updated on save

executions

Field	Type	Required	Description
id	VARCHAR(50)	Yes	Custom ID e.g. exec_8pQr..., PK
job_id	BIGINT	Yes	FK → jobs.id, indexed
status	VARCHAR(20)	Yes	queued running success failed timeout
attempt	INT	Yes	Attempt number. First run = 1
trigger_type	VARCHAR(20)	Yes	scheduled manual retry
started_at	TIMESTAMP	No	When worker picked up the job

completed_at	TIMESTAMP	No	When execution finished
duration_ms	INT	No	Computed: completed_at - started_at
http_status	INT	No	HTTP response code (HTTP jobs only)
error	JSONB	No	{ message, code, stack } on failure
logs	JSONB	No	Array of { ts, level, msg }
next_retry_at	TIMESTAMP	No	When next retry fires
created_at	TIMESTAMP	Yes	DEFAULT NOW()

Redis Key Map

queue:{name}:pending	→ LIST	Pending job IDs (LPUSH to add / RPOP to consume)
queue:{name}:processing	→ SET	Job IDs currently being executed
queue:{name}:failed	→ LIST	Dead letter queue – failed job IDs
job:{id}:lock	→ STRING	Worker heartbeat lock (TTL: 30s, renewed every 10s)
scheduler:next_runs	→ ZSET	{ jobId: nextRunAt_timestamp } – poll this for due jobs
exec:{execId}:log	→ LIST	Real-time log stream during execution
rate:{userId}:trigger	→ STRING	Manual trigger rate limit (max 10/min, TTL: 60s)

2.8 Build Checklist

Task	Priority	Status
☐ Setup PostgreSQL + Redis (Docker Compose)	High	Not Started
☐ Scaffold Spring Boot project with all dependencies in pom.xml	High	Not Started
☐ Setup Flyway migrations — V1__users, V2__jobs, V3__executions	High	Not Started
☐ Implement JWT auth (SecurityConfig + JwtTokenProvider + Filter)	High	Not Started
☐ Build Job CRUD endpoints with full validation (@Valid)	High	Not Started
☐ Build JobPollingTask — @Scheduled every 5s, queries next_run_at	High	Not Started
☐ Build QueueService — Redis LPUSH/RPOP with priority ordering	High	Not Started
☐ Build WorkerService — @Async thread pool picks jobs from queue	High	Not Started
☐ Build HttpJobExecutor — OkHttp with timeout + retry logic	High	Not Started
☐ Build ExecutionService — log each attempt, update status in DB	High	Not Started
☐ Implement dead letter queue — move to failed queue after maxRetry	High	Not Started
☐ Build Dashboard stats endpoint with execution timeline data	Medium	Not Started

☐ Build React dashboard with Recharts execution timeline chart	Medium	Not Started
☐ Add cronstrue human-readable cron display in Job list	Medium	Not Started
☐ Add Swagger UI via springdoc-openapi	Medium	Not Started
☐ k6 load test — trigger 100 jobs simultaneously, record p95	Low	Not Started
☐ Deploy: Railway (Spring Boot + PostgreSQL + Redis)	Low	Not Started

PROJECT 3 — API Gateway with Rate Limiting & Analytics

Register APIs behind a gateway, get per-client rate limiting, usage analytics, latency heatmaps, and anomaly detection. Built with ASP.NET Core + Dapper (raw SQL) + Angular + SignalR.

Why Dapper? Unlike EF Core, Dapper forces you to write raw parameterized SQL — window functions, CTEs, percentile calculations. Interviewers immediately distinguish this from the standard EF Core tutorial project. Interview line: "I used Dapper as a micro-ORM, writing raw SQL including PERCENTILE_CONT window functions for p95/p99 latency analytics."

3.1 Packages to Install

Backend — ASP.NET Core (.csproj)

Package	Version	Purpose
Dapper	2.1.x	Micro-ORM — maps raw SQL results to C# objects
Microsoft.Data.SqlClient	5.2.x	SQL Server connection driver used by Dapper
Microsoft.AspNetCore.Authentication.JwtBearer	8.0.x	JWT Bearer auth middleware
StackExchange.Redis	2.7.x	Redis client — rate limiting + live analytics cache
Microsoft.AspNetCore.SignalR	8.0.x	Real-time analytics push to Angular dashboard
Swashbuckle.AspNetCore	6.5.x	Swagger UI auto-generation
BCrypt.Net-Next	4.0.x	Password hashing for auth
Serilog.AspNetCore	8.0.x	Structured logging
FluentValidation.AspNetCore	11.3.x	Request DTO validation
Yarp.ReverseProxy	2.1.x	Reverse proxy — forward requests to backend services
Hangfire.AspNetCore	1.8.x	Background jobs — flush Redis analytics to SQL
Hangfire.SqlServer	1.8.x	Hangfire SQL Server job storage
Dapper.Contrib	2.0.x	Dapper extension — simple Insert/Update/Delete helpers

```
dotnet add package Dapper
dotnet add package Microsoft.Data.SqlClient
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
dotnet add package StackExchange.Redis
dotnet add package Microsoft.AspNetCore.SignalR
dotnet add package Swashbuckle.AspNetCore
dotnet add package BCrypt.Net-Next
dotnet add package Serilog.AspNetCore
dotnet add package FluentValidation.AspNetCore
dotnet add package Yarp.ReverseProxy
dotnet add package Hangfire.AspNetCore
dotnet add package Hangfire.SqlServer
```

```
dotnet add package Dapper.Contrib
```

Frontend — Angular (npm)

Package	Version	Purpose
@microsoft/signalr	^8.0.0	SignalR client for real-time analytics feed
@angular/material	^17.0.0	UI components — tables, dialogs, progress bars
@ngrx/store	^17.0.0	State management for gateways and analytics
@ngrx/effects	^17.0.0	Side effects — API calls triggered by actions
ng2-charts	^6.0.0	Chart.js wrapper — latency + traffic line charts
chart.js	^4.4.0	Chart.js core (required by ng2-charts)
ngx-toastr	^18.0.0	Toast alerts for rate limit warnings
@swimlane/ngx-datatable	^20.0.0	Paginated sortable request log table
dayjs	^1.11.10	Timestamp formatting throughout dashboard

```
npm install -g @angular/cli
ng new api-gateway-client --routing --style=scss --standalone
cd api-gateway-client
npm install @microsoft/signalr @angular/material @ngrx/store @ngrx/effects \
  ng2-charts chart.js ngx-toastr @swimlane/ngx-datatable dayjs
ng add @angular/material
```

3.2 Environment Variables (appsettings.json)

Variable	Example Value	Description
ConnectionStrings:SqlServer	Server=localhost;Database=ApiGateway;UserId=sa;Password=...	SQL Server connection string — passed directly to Dapper SqlConnection
ConnectionStrings:Redis	localhost:6379	Redis connection string for StackExchange.Redis
JwtSettings:Secret	your_32char_secret_minimum_here__	JWT HMAC-SHA256 signing key
JwtSettings:Issuer	api-gateway.local	JWT issuer claim
JwtSettings:ExpiryDays	7	Token expiry in days
RateLimit:DefaultPerMinute	100	Default rate limit if no per-key rule set
RateLimit:WindowSeconds	60	Sliding window size in seconds
Analytics:FlushIntervalSeconds	10	How often Hangfire flushes Redis counters to SQL
AllowedOrigins	http://localhost:4200	Angular dev URL for CORS policy

3.3 Folder Structure

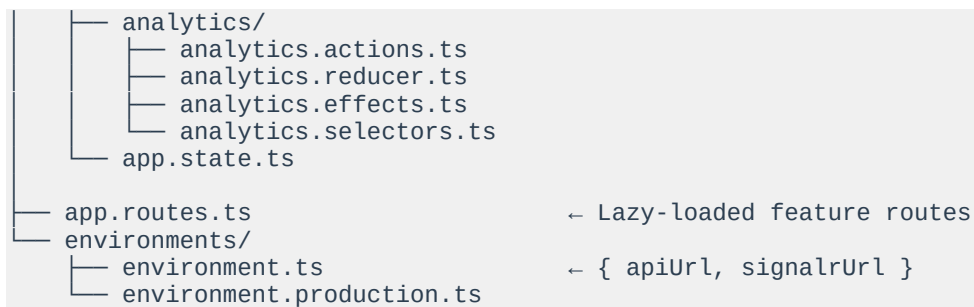
Backend — ASP.NET Core + Dapper

ApiGateway.Api/	
Controllers/	
AuthController.cs	← POST /auth/register, /auth/login, GET /auth/me
GatewayController.cs	← CRUD /gateways
RouteConfigController.cs	← CRUD /gateways/:id/routes
ApiKeyController.cs	← Generate/revoke /api-keys
RateLimitController.cs	← CRUD /rate-limits
AnalyticsController.cs	← GET /analytics/:gatewayId/*
AlertController.cs	← CRUD /alerts
AdminController.cs	← GET /admin/stats
Middleware/	
RateLimitMiddleware.cs	← Redis sliding window per API key
ApiKeyAuthMiddleware.cs	← Validate X-API-Key header → DB lookup
RequestLoggingMiddleware.cs	← Log every proxied request to Redis
AnomalyDetectionMiddleware.cs	← Detect 3x traffic spikes → SignalR alert
Repositories/	
IUserRepository.cs	← ALL Dapper queries live here
UserRepository.cs	← Dapper: INSERT user, SELECT by email/id
IGatewayRepository.cs	
GatewayRepository.cs	← Dapper: CRUD gateways + JOIN routes
IApiKeyRepository.cs	
ApiKeyRepository.cs	← Dapper: INSERT key, SELECT by hash
IRateLimitRepository.cs	
RateLimitRepository.cs	← Dapper: CRUD rate_limit_rules
IAalyticsRepository.cs	
AnalyticsRepository.cs	← Dapper: complex SQL – CTEs, window funcs
RequestLogRepository.cs	← Dapper: bulk INSERT + paginated SELECT
Services/	
AuthService.cs	← BCrypt + JWT generation
GatewayService.cs	← Calls GatewayRepository, builds YARP config
RateLimitService.cs	← Redis ZADD sliding window logic
AnalyticsService.cs	← Aggregates Redis → calls AnalyticsRepository
ApiKeyService.cs	← SHA256 hashing, key generation
AlertService.cs	← Evaluate thresholds, push via SignalR
ProxyService.cs	← YARP dynamic route config
Hubs/	
AnalyticsHub.cs	← SignalR hub – live traffic push every 5s
Models/	
Entities/	← Plain C# classes (no EF attributes)
User.cs	
Gateway.cs	
RouteConfig.cs	
ApiKey.cs	
RateLimitRule.cs	
RequestLog.cs	
Alert.cs	
DTOs/	
Request/	
Response/	
Database/	
DbConnectionFactory.cs	← Returns new SqlConnection(connString)
Migrations/	← Plain .sql files, run manually or via DbUp
V1_create_users.sql	
V2_create_gateways.sql	
V3_create_route_configs.sql	
V4_create_api_keys.sql	

└─ V5__create_rate_limit_rules.sql └─ V6__create_request_logs.sql └─ V7__create_alerts.sql	
─ BackgroundJobs/ └─ AnalyticsFlushJob.cs └─ DailyReportJob.cs	└─ Hangfire: flush Redis counters → SQL (10s) └─ Hangfire: daily usage summary per gateway
─ Validators/	└─ FluentValidation rules per DTO
─ Extensions/ └─ ServiceExtensions.cs └─ MiddlewareExtensions.cs	└─ DI registrations in one place └─ Middleware pipeline helpers
─ appsettings.json ─ appsettings.Development.json ─ Program.cs	└─ Middleware pipeline order matters here

Frontend — Angular

api-gateway-client/src/app/	
─ core/ └─ interceptors/ └─ auth.interceptor.ts └─ error.interceptor.ts └─ guards/ └─ auth.guard.ts └─ services/ └─ signalr.service.ts	└─ Attach JWT Bearer to every HTTP request └─ Handle 401 redirect, 429 rate limit toast └─ Protect routes, redirect to login └─ Connect to hub, expose event Observables
─ features/ └─ auth/ └─ login/login.component.ts └─ register/register.component.ts └─ dashboard/ └─ dashboard.component.ts └─ traffic-chart/ └─ latency-heatmap/ └─ gateways/ └─ gateway-list/ └─ gateway-detail/ └─ gateway-form/ └─ analytics/ └─ analytics.component.ts └─ request-log-table/ └─ error-rate-chart/ └─ rate-limits/ └─ rate-limit-list/ └─ rate-limit-form/ └─ alerts/ └─ alert-list/ └─ alert-form/	└─ Summary cards + live traffic chart └─ ng2-charts realtime line chart └─ P50/P95/P99 per route display └─ Table with status, requests today, error rate └─ Routes, keys, rate limit rules per gateway └─ Create / edit gateway └─ Per-gateway traffic breakdown └─ ngx-datatable paginated logs └─ 4xx/5xx error rate over time └─ All rules with usage progress bars └─ Create / edit rules └─ Active + triggered alerts └─ Set anomaly thresholds
─ shared/ └─ components/ └─ stat-card/ └─ status-badge/ └─ copy-button/ └─ pipes/ └─ format-ms.pipe.ts	└─ Reusable metric card with trend arrow └─ Color-coded active/paused/blocked badge └─ Copy API key to clipboard └─ Format latency values e.g. 94ms, 1.2s
─ store/ └─ gateway/ └─ gateway.actions.ts └─ gateway.reducer.ts └─ gateway.effects.ts └─ gateway.selectors.ts	└─ API calls triggered by actions



3.4 Endpoint Overview

Method	Endpoint	Auth	Description
POST	/api/auth/register	Public	Register new user account
POST	/api/auth/login	Public	Login and receive JWT
GET	/api/auth/me	JWT	Current user profile
GET	/api/gateways	JWT	List all gateways for user
POST	/api/gateways	JWT	Register new gateway
GET	/api/gateways/:id	JWT	Gateway detail with routes and keys
PUT	/api/gateways/:id	JWT	Update gateway config
DELETE	/api/gateways/:id	JWT	Remove gateway
GET	/api/gateways/:id/routes	JWT	List proxy routes for gateway
POST	/api/gateways/:id/routes	JWT	Add proxy route
PUT	/api/gateways/:id/ routes/:routeId	JWT	Update route
DELETE	/api/gateways/:id/ routes/:routeId	JWT	Remove route
GET	/api/api-keys	JWT	List all API keys
POST	/api/api-keys	JWT	Generate new API key
DELETE	/api/api-keys/:keyId	JWT	Revoke API key
GET	/api/rate-limits	JWT	List all rate limit rules
POST	/api/rate-limits	JWT	Create rate limit rule
PUT	/api/rate-limits/:id	JWT	Update rule
DELETE	/api/rate-limits/:id	JWT	Delete rule
GET	/api/ analytics/:gatewayId	JWT	Traffic summary stats
GET	/api/ analytics/:gatewayId/ latency	JWT	P50/P95/P99 latency per route
GET	/api/ analytics/:gatewayId/ errors	JWT	4xx/5xx error rate over time

GET	/api/analytics/:gatewayId/requests	JWT	Paginated request log
GET	/api/alerts	JWT	List alert rules
POST	/api/alerts	JWT	Create anomaly alert rule
GET	/api/admin/stats	JWT	Platform-wide stats (super admin)
GET	/api/health	Public	Health check: SQL + Redis + YARP

3.5 Key Endpoint Details

| [POST] /api/gateways *Register a new API gateway*

Auth: JWT required

Request Body:

```
{
  "name": "Payment Service Gateway",
  "targetBaseUrl": "https://payments.internal:8080",
  "defaultRateLimitPerMinute": 100,
  "routes": [
    { "path": "/payments/{**catch-all}", "methods": ["GET","POST"], "stripPrefix":
false },
    { "path": "/refunds/{**catch-all}", "methods": ["POST"], "stripPrefix":
true }
  ]
}
```

Response 201: { "id": 1, "gatewayId": "gw_pYmX9kQr", "status": "active" }

| [POST] /api/api-keys *Generate API key — returned once, never again*

Request Body: { "gatewayId": 1, "label": "Production App", "expiresAt": "2026-01-01" }

Response 201:

```
{
  "id": 5,
  "key": "sk_live_xK9mP2qRtYhJ8nWs",
  "keyPrefix": "sk_live_xK9m",
  "label": "Production App",
  "warning": "Store this key securely. It will never be shown again."
}
```

Dapper insert: Store SHA256(key) in DB. Return plaintext only once. On every request, hash incoming key and SELECT WHERE key_hash = @hash.

| [POST] /api/rate-limits *Create rate limit rule*

Request Body:

```
{
  "gatewayId": 1,
  "scope": "api-key",
  "apiKeyId": 5,
  "requestsPerWindow": 200,
  "windowSeconds": 60,
  "algorithm": "sliding-window",
  "burstAllowance": 50
}
```

scope: "api-key" | "ip" | "gateway"

algorithm: "sliding-window" | "token-bucket" | "fixed-window"

| **[GET] /api/analytics/:gatewayId/latency** *P50/P95/P99 via SQL window functions*

Dapper query used:

```
-- This is the actual SQL Dapper runs (window function)
SELECT
  path,
  PERCENTILE_CONT(0.50) WITHIN GROUP (ORDER BY latency_ms) AS p50,
  PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY latency_ms) AS p95,
  PERCENTILE_CONT(0.99) WITHIN GROUP (ORDER BY latency_ms) AS p99,
  COUNT(*) AS total_requests
FROM request_logs
WHERE gateway_id = @GatewayId
  AND timestamp >= @From AND timestamp <= @To
GROUP BY path
ORDER BY total_requests DESC;
```

Response 200:

```
{
  "latency": { "p50": 45, "p75": 98, "p95": 210, "p99": 480 },
  "byRoute": [
    { "path": "/payments/**", "p50": 42, "p95": 190, "p99": 420, "requests": 32000 },
    { "path": "/refunds/**", "p50": 89, "p95": 310, "p99": 680, "requests": 16200 }
  ]
}
```

| **[GET] /api/analytics/:gatewayId/requests** *Paginated request log with CTE*

Dapper query used:

```
-- CTE for pagination with total count in one query
WITH filtered AS (
  SELECT r.*, k.label AS api_key_label,
         ROW_NUMBER() OVER (ORDER BY r.timestamp DESC) AS rn,
         COUNT(*) OVER () AS total_count
  FROM request_logs r
  LEFT JOIN api_keys k ON k.id = r.api_key_id
  WHERE r.gateway_id = @GatewayId
        AND (@StatusCode IS NULL OR r.status_code = @StatusCode)
        AND r.timestamp BETWEEN @From AND @To
)
SELECT * FROM filtered
WHERE rn BETWEEN @Offset + 1 AND @Offset + @Limit;
```

3.6 SignalR Hub — Real-Time Analytics

| *Angular connects to /hubs/analytics?access_token=JWT. Hub broadcasts live metrics every 5 seconds from Redis without hitting SQL.*

```
// Server → Angular (subscribe in signalr.service.ts)
ReceiveTrafficUpdate → { gatewayId, requestsPerMin, errorRate, avgLatencyMs }
ReceiveRateLimitAlert → { gatewayId, apiKeyPrefix, clientId, blockedCount }
ReceiveAnomalyAlert → { gatewayId, type: 'spike'|'error_surge', message, severity }

// Angular → Server
JoinGateway(gatewayId) → Subscribe to real-time events for this gateway
LeaveGateway(gatewayId) → Unsubscribe
```

3.7 Database Tables — SQL Server

Dapper note: No EF Core. No DbContext. No migrations auto-generated. You write plain .sql files and run them manually (or via DbUp). Every query in repositories is a raw SQL string passed to Dapper's `Query<T>()` or `Execute()`.

Table: Users

Field	Type	Required	Description
Id	BIGINT	Yes	PRIMARY KEY IDENTITY(1,1)
Name	NVARCHAR(100)	Yes	Full display name
Email	NVARCHAR(255)	Yes	UNIQUE, used for login. INDEX: IX_Users_Email
PasswordHash	NVARCHAR(500)	Yes	BCrypt hashed — never store plaintext
Role	NVARCHAR(20)	Yes	user admin. DEFAULT 'user'
CreatedAt	DATETIME2	Yes	DEFAULT GETUTCDATE()
UpdatedAt	DATETIME2	Yes	Updated on every save

```
CREATE TABLE Users (  
  Id          BIGINT          PRIMARY KEY IDENTITY(1,1),  
  Name       NVARCHAR(100) NOT NULL,  
  Email      NVARCHAR(255) NOT NULL,  
  PasswordHash NVARCHAR(500) NOT NULL,  
  Role       NVARCHAR(20)  NOT NULL DEFAULT 'user',  
  CreatedAt  DATETIME2     NOT NULL DEFAULT GETUTCDATE(),  
  UpdatedAt  DATETIME2     NOT NULL DEFAULT GETUTCDATE()  
);  
CREATE UNIQUE INDEX IX_Users_Email ON Users(Email);
```

Table: Gateways

Field	Type	Required	Description
Id	BIGINT	Yes	PRIMARY KEY IDENTITY(1,1)
UserId	BIGINT	Yes	FK → Users.Id ON DELETE CASCADE. INDEX: IX_Gateways_UserId
GatewayUid	NVARCHAR(50)	Yes	Custom ID e.g. gw_pYmX9kQr. UNIQUE
Name	NVARCHAR(200)	Yes	Display name
Description	NVARCHAR(500)	No	Optional description
TargetBaseUrl	NVARCHAR(500)	Yes	Backend service base URL
Status	NVARCHAR(20)	Yes	active paused deleted. DEFAULT 'active'
DefaultRateLimitPer	INT	No	Fallback rate limit. DEFAULT 100

Min			
CreatedAt	DATETIME2	Yes	DEFAULT GETUTCDATE()
UpdatedAt	DATETIME2	Yes	Updated on every save

```
CREATE TABLE Gateways (
  Id BIGINT PRIMARY KEY IDENTITY(1,1),
  UserId BIGINT NOT NULL,
  GatewayUid NVARCHAR(50) NOT NULL,
  Name NVARCHAR(200) NOT NULL,
  Description NVARCHAR(500) NULL,
  TargetBaseUrl NVARCHAR(500) NOT NULL,
  Status NVARCHAR(20) NOT NULL DEFAULT 'active',
  DefaultRateLimitPerMin INT NULL DEFAULT 100,
  CreatedAt DATETIME2 NOT NULL DEFAULT GETUTCDATE(),
  UpdatedAt DATETIME2 NOT NULL DEFAULT GETUTCDATE(),
  CONSTRAINT FK_Gateways_Users FOREIGN KEY (UserId)
    REFERENCES Users(Id) ON DELETE CASCADE
);
CREATE UNIQUE INDEX IX_Gateways_GatewayUid ON Gateways(GatewayUid);
CREATE INDEX IX_Gateways_UserId ON Gateways(UserId);
```

Table: RouteConfigs

Field	Type	Required	Description
Id	BIGINT	Yes	PRIMARY KEY IDENTITY(1,1)
GatewayId	BIGINT	Yes	FK → Gateways.Id ON DELETE CASCADE. INDEX: IX_RouteConfigs_GatewayId
Path	NVARCHAR(500)	Yes	Route pattern e.g. /payments/{**catch-all}
Methods	NVARCHAR(100)	Yes	Comma-separated HTTP methods: GET,POST,PUT
StripPrefix	BIT	Yes	Remove path prefix before forwarding. DEFAULT 0
IsActive	BIT	Yes	DEFAULT 1
CreatedAt	DATETIME2	Yes	DEFAULT GETUTCDATE()

```
CREATE TABLE RouteConfigs (
  Id BIGINT PRIMARY KEY IDENTITY(1,1),
  GatewayId BIGINT NOT NULL,
  Path NVARCHAR(500) NOT NULL,
  Methods NVARCHAR(100) NOT NULL,
  StripPrefix BIT NOT NULL DEFAULT 0,
  IsActive BIT NOT NULL DEFAULT 1,
  CreatedAt DATETIME2 NOT NULL DEFAULT GETUTCDATE(),
  CONSTRAINT FK_RouteConfigs_Gateways FOREIGN KEY (GatewayId)
    REFERENCES Gateways(Id) ON DELETE CASCADE
);
CREATE INDEX IX_RouteConfigs_GatewayId ON RouteConfigs(GatewayId);
```

Table: ApiKeys

Field	Type	Required	Description
Id	BIGINT	Yes	PRIMARY KEY IDENTITY(1,1)
GatewayId	BIGINT	Yes	FK → Gateways.Id ON DELETE CASCADE. INDEX: IX_ApiKeys_GatewayId
KeyHash	NVARCHAR(500)	Yes	SHA256 hash of actual key. UNIQUE. Never store plaintext.
KeyPrefix	NVARCHAR(20)	Yes	First 12 chars for display e.g. sk_live_xK9m
Label	NVARCHAR(100)	No	Human label e.g. Production App
IsActive	BIT	Yes	DEFAULT 1. Set to 0 on revoke.
ExpiresAt	DATETIME2	No	NULL means never expires
LastUsedAt	DATETIME2	No	Updated on every authenticated request
CreatedAt	DATETIME2	Yes	DEFAULT GETUTCDATE()

```
CREATE TABLE ApiKeys (  
  Id          BIGINT          PRIMARY KEY IDENTITY(1,1),  
  GatewayId   BIGINT          NOT NULL,  
  KeyHash     NVARCHAR(500)   NOT NULL,  
  KeyPrefix   NVARCHAR(20)    NOT NULL,  
  Label       NVARCHAR(100)   NULL,  
  IsActive    BIT             NOT NULL DEFAULT 1,  
  ExpiresAt   DATETIME2       NULL,  
  LastUsedAt  DATETIME2       NULL,  
  CreatedAt   DATETIME2       NOT NULL DEFAULT GETUTCDATE(),  
  CONSTRAINT FK_ApiKeys_Gateways FOREIGN KEY (GatewayId)  
    REFERENCES Gateways(Id) ON DELETE CASCADE  
);  
CREATE UNIQUE INDEX IX_ApiKeys_KeyHash ON ApiKeys(KeyHash);  
CREATE INDEX IX_ApiKeys_GatewayId ON ApiKeys(GatewayId);
```

Table: RateLimitRules

Field	Type	Required	Description
Id	BIGINT	Yes	PRIMARY KEY IDENTITY(1,1)
GatewayId	BIGINT	Yes	FK → Gateways.Id ON DELETE CASCADE. INDEX: IX_RateLimitRules_GatewayId
Scope	NVARCHAR(20)	Yes	api-key ip gateway
ApiKeyId	BIGINT	No	FK → ApiKeys.Id ON DELETE SET NULL. NULL when scope = ip or gateway
RequestsPerWindow	INT	Yes	Max requests in the window
WindowSeconds	INT	Yes	Rolling window size in seconds
Algorithm	NVARCHAR(30)	Yes	sliding-window token-bucket fixed-window
BurstAllowance	INT	No	Extra burst capacity. DEFAULT 0

IsActive	BIT	Yes	DEFAULT 1
CreatedAt	DATETIME2	Yes	DEFAULT GETUTCDATE()

```
CREATE TABLE RateLimitRules (
  Id BIGINT PRIMARY KEY IDENTITY(1,1),
  GatewayId BIGINT NOT NULL,
  Scope NVARCHAR(20) NOT NULL,
  ApiKeyId BIGINT NULL,
  RequestsPerWindow INT NOT NULL,
  WindowSeconds INT NOT NULL,
  Algorithm NVARCHAR(30) NOT NULL DEFAULT 'sliding-window',
  BurstAllowance INT NOT NULL DEFAULT 0,
  IsActive BIT NOT NULL DEFAULT 1,
  CreatedAt DATETIME2 NOT NULL DEFAULT GETUTCDATE(),
  CONSTRAINT FK_RateLimitRules_Gateways FOREIGN KEY (GatewayId)
    REFERENCES Gateways(Id) ON DELETE CASCADE,
  CONSTRAINT FK_RateLimitRules_ApiKeys FOREIGN KEY (ApiKeyId)
    REFERENCES ApiKeys(Id) ON DELETE SET NULL
);
CREATE INDEX IX_RateLimitRules_GatewayId ON RateLimitRules(GatewayId);
```

Table: RequestLogs

Performance note: This table grows fast — every proxied request = one row. INDEX on (GatewayId, Timestamp) is critical for analytics queries. Partition by month in production.

Field	Type	Required	Description
Id	NVARCHAR(50)	Yes	Custom ID e.g. req_8Xp... PRIMARY KEY — avoids IDENTITY lock contention
GatewayId	BIGINT	Yes	FK → Gateways.Id ON DELETE CASCADE. COMPOSITE INDEX with Timestamp
ApiKeyId	BIGINT	No	FK → ApiKeys.Id ON DELETE SET NULL. NULL for unauthenticated
Method	NVARCHAR(10)	Yes	GET POST PUT DELETE PATCH
Path	NVARCHAR(500)	Yes	Full request path e.g. /payments/charge
StatusCode	INT	Yes	HTTP response status from backend service
LatencyMs	INT	Yes	Time from gateway receive to response sent
ClientIp	NVARCHAR(45)	Yes	IPv4 or IPv6 client address
IsRateLimited	BIT	Yes	1 if returned 429, 0 otherwise. DEFAULT 0
Timestamp	DATETIME2	Yes	When request was received. COMPOSITE INDEX with GatewayId

```
CREATE TABLE RequestLogs (
  Id NVARCHAR(50) PRIMARY KEY,
  GatewayId BIGINT NOT NULL,
  ApiKeyId BIGINT NULL,
  Method NVARCHAR(10) NOT NULL,
  Path NVARCHAR(500) NOT NULL,
  StatusCode INT NOT NULL,
```

```

    LatencyMs          INT          NOT NULL,
    ClientIp           NVARCHAR(45) NOT NULL,
    IsRateLimited      BIT          NOT NULL DEFAULT 0,
    Timestamp           DATETIME2    NOT NULL DEFAULT GETUTCDATE(),
    CONSTRAINT FK_RequestLogs_Gateways FOREIGN KEY (GatewayId)
        REFERENCES Gateways(Id) ON DELETE CASCADE,
    CONSTRAINT FK_RequestLogs_ApiKeys FOREIGN KEY (ApiKeyId)
        REFERENCES ApiKeys(Id) ON DELETE SET NULL
);
-- CRITICAL: composite index for analytics queries
CREATE INDEX IX_RequestLogs_GatewayId_Timestamp
    ON RequestLogs(GatewayId, Timestamp DESC);

```

Table: Alerts

Field	Type	Required	Description
Id	BIGINT	Yes	PRIMARY KEY IDENTITY(1,1)
GatewayId	BIGINT	Yes	FK → Gateways.Id ON DELETE CASCADE
Name	NVARCHAR(200)	Yes	Alert display name
MetricType	NVARCHAR(50)	Yes	error_rate latency_p95 traffic_spike rate_limit_hit
ThresholdValue	DECIMAL(10, 2)	Yes	Trigger value e.g. 5.0 for 5% error rate
ThresholdUnit	NVARCHAR(20)	Yes	percent ms requests_per_min
IsActive	BIT	Yes	DEFAULT 1
LastTriggeredAt	DATETIME2	No	NULL if never triggered
CreatedAt	DATETIME2	Yes	DEFAULT GETUTCDATE()

```

CREATE TABLE Alerts (
    Id          BIGINT          PRIMARY KEY IDENTITY(1,1),
    GatewayId   BIGINT          NOT NULL,
    Name        NVARCHAR(200)   NOT NULL,
    MetricType  NVARCHAR(50)    NOT NULL,
    ThresholdValue DECIMAL(10,2) NOT NULL,
    ThresholdUnit NVARCHAR(20)  NOT NULL,
    IsActive    BIT             NOT NULL DEFAULT 1,
    LastTriggeredAt DATETIME2    NULL,
    CreatedAt    DATETIME2       NOT NULL DEFAULT GETUTCDATE(),
    CONSTRAINT FK_Alerts_Gateways FOREIGN KEY (GatewayId)
        REFERENCES Gateways(Id) ON DELETE CASCADE
);

```

3.8 Table Relationships

Read this: Every foreign key and its cascade behavior is defined here. Know this section cold — interviewers ask about cascade deletes and NULL FK choices.

Users (1) ————— (N) Gateways
 One user owns many gateways
 FK: Gateways.UserId → Users.Id

ON DELETE CASCADE → delete user = delete all their gateways

Gateways (1) — (N) RouteConfigs
One gateway has many proxy routes
FK: RouteConfigs.GatewayId → Gateways.Id
ON DELETE CASCADE → delete gateway = delete all its routes

Gateways (1) — (N) ApiKeys
One gateway has many API keys
FK: ApiKeys.GatewayId → Gateways.Id
ON DELETE CASCADE → delete gateway = delete all its keys

Gateways (1) — (N) RateLimitRules
One gateway has many rate limit rules
FK: RateLimitRules.GatewayId → Gateways.Id
ON DELETE CASCADE

ApiKeys (1) — (N) RateLimitRules
One API key can have its own rate limit rule
FK: RateLimitRules.ApiKeyId → ApiKeys.Id
ON DELETE SET NULL → delete key = rule stays but becomes gateway-scoped

Gateways (1) — (N) RequestLogs
One gateway logs many requests
FK: RequestLogs.GatewayId → Gateways.Id
ON DELETE CASCADE

ApiKeys (1) — (N) RequestLogs
Requests optionally linked to the key used
FK: RequestLogs.ApiKeyId → ApiKeys.Id
ON DELETE SET NULL → delete key = logs stay, ApiKeyId becomes NULL

Gateways (1) — (N) Alerts
One gateway has many alert rules
FK: Alerts.GatewayId → Gateways.Id
ON DELETE CASCADE

Why ON DELETE SET NULL for RequestLogs.ApiKeyId?

When a key is revoked, historical request logs must NOT be deleted — they're audit data. SET NULL preserves the log row but clears the key reference. This is an interview question waiting to happen.

Redis Key Map

ratelimit:{gatewayId}:{apiKeyId}:{windowTs}	→ ZSET	Sliding window timestamps per key
ratelimit:{gatewayId}:ip:{hashedIp}:{ts}	→ ZSET	Sliding window per client IP
gateway:{gatewayId}:stats:live last 60s	→ HASH	{ requests, errors, rateLimited }
gateway:{gatewayId}:latency percentile	→ LIST	Last 1000 latency values for
apikey:{keyHash}:cache DB hit)	→ STRING	Gateway config (TTL: 5min, avoids
anomaly:{gatewayId}:baseline detection	→ HASH	Rolling avg requests/min for spike

3.9 Build Checklist

Task	Priority	Status
☐ Create SQL Server DB and run all 7 migration .sql files in	High	Not Started

order		
☐ Setup DbConnectionFactory — returns new SqlConnection(connString)	High	Not Started
☐ Implement JWT auth middleware (JwtBearer + token generation service)	High	Not Started
☐ Build UserRepository with Dapper — INSERT + SELECT by email	High	Not Started
☐ Build GatewayRepository — CRUD + JOIN with RouteConfigs in one query	High	Not Started
☐ Build ApiKeyAuthMiddleware — hash X-API-Key header → Dapper SELECT	High	Not Started
☐ Build RateLimitMiddleware — Redis ZADD sliding window per key	High	Not Started
☐ Setup YARP reverse proxy — forward requests to gateway.TargetBaseUrl	High	Not Started
☐ Build RequestLoggingMiddleware — push to Redis LIST after each request	High	Not Started
☐ Build RequestLogRepository — bulk INSERT + CTE paginated SELECT	High	Not Started
☐ Build AnalyticsRepository — PERCENTILE_CONT window function query	High	Not Started
☐ Build AnalyticsHub (SignalR) — push Redis stats to Angular every 5s	High	Not Started
☐ Setup Hangfire — flush Redis request buffer → SQL every 10s	High	Not Started
☐ Build AnomalyDetectionMiddleware — detect 3x baseline traffic spikes	Medium	Not Started
☐ Build AlertService — evaluate thresholds + fire SignalR alert event	Medium	Not Started
☐ Scaffold Angular with NgRx store, routing, auth interceptor	Medium	Not Started
☐ Build Angular SignalR service — subscribe to hub events as Observables	Medium	Not Started
☐ Build Dashboard — live traffic chart + latency P95/P99 per route	Medium	Not Started
☐ Build request log table — ngx-datatable with filters + pagination	Medium	Not Started
☐ Build rate limit rules UI — current usage bars + create/edit form	Medium	Not Started
☐ Add Swagger UI (Swashbuckle) with JWT Bearer scheme configured	Low	Not Started
☐ Deploy: Azure App Service + Azure SQL + Azure Cache for Redis	Low	Not Started